

ECE 757 Project Report: Compressed Decoupled Z-Cache

Sreeraj Kannakarankodi, Noah Madison Scott, Tudor Fabian Popa,
Vidhyanandhan Sathish Kumar, Abhinarayani Velu Radhakrishnan

Abstract

We propose a unification of the Z-Cache and Bit-Plane Cache Compression (BPC) systems, such that the disadvantages of Z-Cache, that are normally solved by increasing the cache size, are addressed by the compression solution.

There is an inherent incompatibility between the standard implementation of Z-Cache and most cache compression algorithms due to the variable block size. Thus we propose a decoupled design that attempts to cleanly solve this incompatibility without invoking major modifications to the Z-Cache and BPC systems.

Ultimately we find that our solution performs very well on easily compressible data, such as during financial simulations. But in the cases of scientific or heavy computation benchmarks, which have low data similarity across a cache line, compress poorly, and our implementation performs worse than baseline.

Motivation

Z-Cache virtually expands the associativity of a cache by 'hijacking' the ways of other addresses during a cache miss and victim selection. It selects which lines to hijack by modifying the skewed cache design, each line is selected by a different hash function. Z-Cache's effectiveness relies on the fact that the probability that its virtual way system interferes with important cache lines is rather low[1]. The chance of this destructive interference lowers with larger and larger caches.

It then behooves us to want to give Z-Cache as large a cache as possible to work with. The LLC is a prime candidate for this, but doubling the LLC freely is not always possible. Depending on the power or die budget.

We then propose that the solution lies in cache compression. By applying compression to the cache, we can emulate a larger cache with only a fraction of the die space and power consumption. If some cache compression technique provides a 2:1 compression ratio, then we'd expect the new Z-Cache to perform equivalent to a Z-Cache with double the cache capacity.

Challenges

For our compression algorithm choice, we chose Bit-Plane Cache Compression[2]. Chosen due to its ease of imple-

mentation and simple design. We'd had hoped to use a delta-style compressor, like xor-cache[3], but time did not allow.

Our goal for the design was to provide something that required little to no changes to BPC or Z-Cache. Most of the logic needed to be implemented in a 'glue' module, rather than worked into the other components. This was to make the design more attractive, as it would minimize the difficulty of verification.

The first major issue that we faced was an inherent incompatibility between a compressed cache and Z-Cache. Z-Cache performs 'cuckoo cache' style moves between selected possible-victim blocks, selected in a 'victim-tree'[1, 4]. This means, for any given victim-tree constructed, all victims that will undergo this cuckoo shuffle have to be the same size. This is a fundamental force against compression, which will produce blocks of varying sizes depending on how effective the compression was for that data.

To both fulfill our design goals and solve the inherent incompatibility of the design, we had to consider decoupled designs. Systems that isolated each component from the rest of the system as much as possible, such that we can attempt to solve these incompatibilities with deferment and abstraction.

Design

We propose a Compressed Decoupled Z-Cache (CDZ-Cache) system [fig 1]. Our design can be split into three major parts: the compressor system, the data allocation system, and the Z-Cache system.

The BPC compressor system has been slightly modified to restrain the variation in block sizes it can return. It has been restricted to the sizes of 8 bytes, 16 bytes, 32 bytes, and 64 bytes. The size of the compressed block is padded to the nearest restricted size before it is sent to the data allocation system.

The data allocation system is the 'glue' that brings the BPC and Z-cache systems together. It is composed of two parts: a data array, and a buddy-allocator system. Blocks are sent from the compressor to the system, which will then allocate room for them in the data array. The resulting pointer to the allocation is then sent to the Z-cache. If the data array suffers external fragmentation to the point that an allocation fails, it will trigger a defragmentation eviction.

The chance of a buddy allocator, managing four block sizes, to experience external fragmentation is low, but to address the corner case we implement a pseudo-random defragmentation policy. Every couple cycles, one of four pointers, one for each block size, will access a pseudo-random address (generated with a linear-feedback shift register). If this address is of that pointer's appropriate block size, the pointer will hold it forward. Once a defragmentation event happens, the pointer of the block size needed will trigger a 'silent' eviction of that block and notify the Z-Cache. This eviction is silent, as it does not invoke the Z-Cache policy logic, it is a simple forced writeback-and-invalidate.

The Z-Cache system had been slightly modified to now hold the pointers returned from the data array as its basic cache block, rather than any actual data. This means the skewing and cuckoo policy logic can remain completely unchanged, while still managing variable sized blocks at a higher level. As each block is managed by a fixed sized pointer, solving the incompatibility.

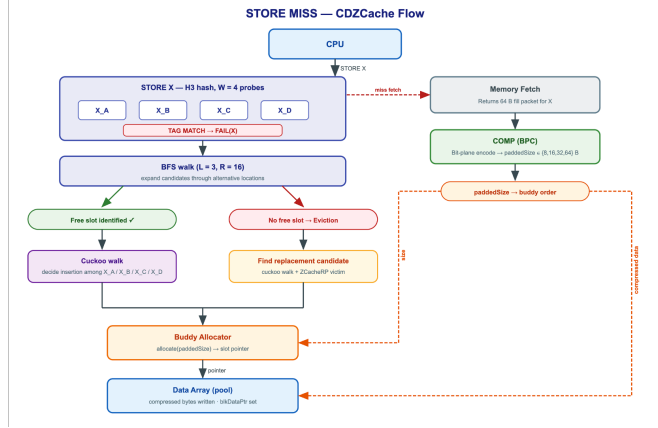


Figure 1: High-level CDZ-Cache diagram

Results

We implemented CDZ-Cache as a Gem5 LLC policy[5, 6]. It was benchmarked on the PARSEC 2.1 benchmark suite[7, 8], and compared to a normal Z-Cache of equivalent size and one of double size as baselines. Gem5 was set to use a dual core Timing Simple CPU with a 32 KB L1 per-core cache, and a shared 256 KB L2 cache, which acted as the LLC. In the case of the second baseline, the L2 size was doubled.

As seen in **Fig. 2**, our implementation performs just as well if not better than a double capacity Z-Cache under Swaptions and achieves halfway the performance of the double capacity benchmark under Blacksholes. However, it was only able to match the base benchmark under Streamcluster and Canneal, and it performed significantly worse under Fluidanimate and Freqmine.

The effectiveness of our implementation seems to rely heavily on the effectiveness of the BPC implementation. Our compression implementation only helps when the application's data has numerical regularity. For the two financial workloads (blacksholes and swaptions) there is a clear advantage. For simulation and data-mining workloads with diverse or irregular data, BPC performs very poorly, and the compression and decoupling system only seem to hold the performance of standard Z-Cache back, rather than enhance it.

L2 Cache Performance Across PARSEC Workloads

CDZ (256 KiB, BDI compression, no prefetch) Z-Cache B1 (256 KiB, stride prefetch) Z-Cache×2 B2 (512 KiB, stride prefetch)

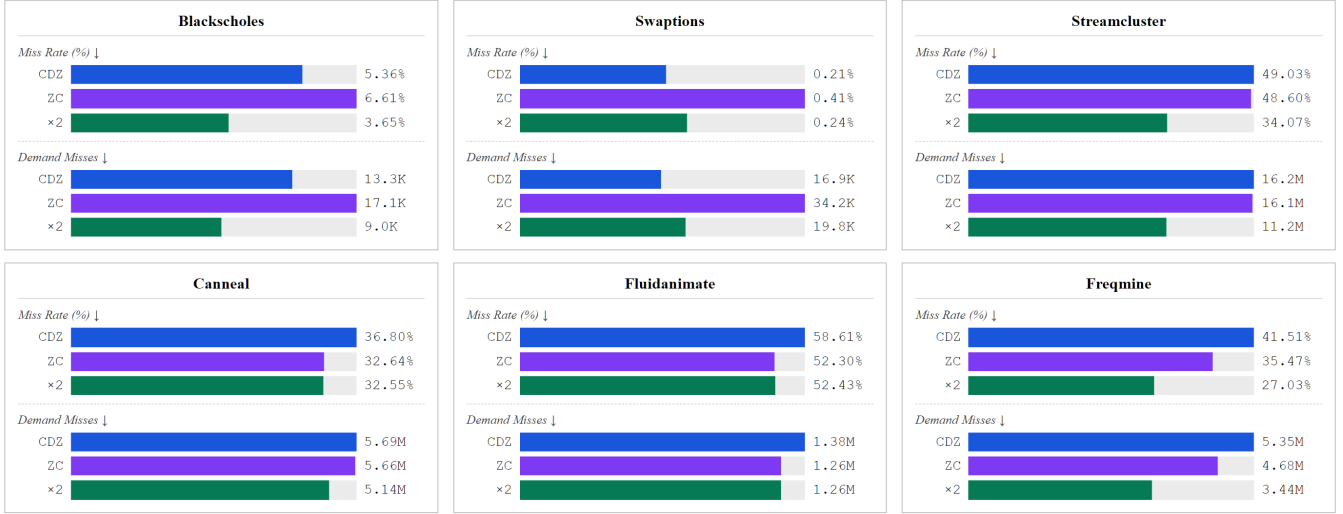


Figure 2: L2 miss rate (%) and demand miss count for three cache configurations across six PARSEC workloads, simulated with gem5 full-system simulation (x86, 2-core, 16-way L2, 1 THz simulated frequency). **CDZ:** Z-Cache (256 KiB) augmented with BDI compression. **Z-Cache B1:** Z-Cache baseline (256 KiB). **Z-Cache×2 B2:** Z-Cache (512 KiB). Bar widths are normalized to the worst-performing configuration per metric per workload. Area overhead: CDZ $\approx 1.15\times$, Z-Cache B1 = $1.00\times$ (reference), Z-Cache×2 B2 $\approx 1.90\times$. K = thousands; M = millions.

Future work

First and foremost, BPC is not an all-around high performing compression algorithm; it was chosen for its simplicity and ease of implementation. BPC struggles with float compression, and completely fails when the data within a cache line has low self-similarity, such as during graph exploration on large graphs. A major point of future work is to choose a compression algorithm with far better characteristics. CDZ-Cache’s effectiveness with Swaptions shows when the compression algorithm aligns with the nature of the data, we can approach the performance of a simple Z-Cache twice as large.

The decoupled nature of the system might allow for multiple compression algorithms that feed into the same data allocation system, with the smallest block chosen for storage, and only a few additional bits per allocation to store the compression type. Compression logic generally consumes less die space than SRAM, and a convergence point where performance expected for a cache twice as large, for general purpose CPU applications, in a die space half as large, may be possible.

Additionally, our solution for de-fragmentation of the data array is by-and-large a naive band-aid. The pseudo-random policy was chosen for its simplicity rather than any theory of effectiveness, and relies on the fact that a buddy allocator, that only has to shuffle 4 sizes all powers of two, has a very low chance of external fragmentation. Our solution was mainly to avoid corner case errors rather than be any permanent solution. A much more intelligent approach to de-fragmentation would be future work.

Finally, we had planned a system for ‘sticky’ cache

lines/segments early in our project, but abandoned it out of time constraints. For future work, our design would involve a LRU counter modifier that uses the cache line’s compression level to nudge highly compressed lines closer to the future, making them less likely to be chosen as a victim by the replacement policy. This would not be ideal for every policy, highly complex ones such as mockingjay[9] might lose performance, but would improve compression performance, and would likely work fine with Z-Cache.

References

- [1] Daniel Sanchez and Christos Kozyrakis. The ZCache: Decoupling ways and associativity. In *43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 187–198, 2010. doi: 10.1109/MICRO.2010.20.
- [2] Jungrae Kim, Michael Sullivan, Esha Choukse, and Mattan Erez. Bit-plane compression: Transforming data for better compression in many-core architectures. In *43rd ACM/IEEE International Symposium on Computer Architecture (ISCA)*, pages 329–340, 2016. doi: 10.1109/ISCA.2016.37.
- [3] Zhewen Pan and Joshua San Miguel. The XOR cache: A catalyst for compression. In *52nd ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2025. doi: 10.1145/3695053.3730995.
- [4] Rasmus Pagh and Flemming Friche Rodler. Cuckoo

- p hashing.
- Journal of Algorithms*
- , 51(2):122–144, 2004. doi: 10.1016/j.jalgor.2003.12.002.
- [5] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011. doi: 10.1145/2024716.2024718.
 - [6] Jason Lowe-Power et al. The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152*, 2020.
 - [7] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. The PARSEC benchmark suite: Characterization and architectural implications. In *17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 72–81, 2008. doi: 10.1145/1454115.1454128.
 - [8] Christian Bienia. *Benchmarking Modern Multiprocessors*. PhD thesis, Princeton University, 2011.
 - [9] Ishan Shah, Akanksha Jain, and Calvin Lin. Effective mimicry of Belady’s MIN policy. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 558–572, 2022. doi: 10.1109/HPCA53966.2022.00048.